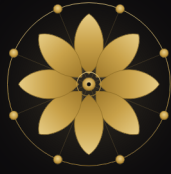


SOVEREIGN · GOVERNED · SELF-HOSTABLE



The Sovereign Agentic OS

One governed operating system for data,
knowledge, and agents.

Data Masterclass • datamasterclass.com

An introduction • July 2026

Contents

Why the Sovereign Agentic OS exists

Governance by default

The two ideas you must know

How the OS is organised

Entry — the front doors

Plan — set direction

Context — govern what agents may use

Build — create value on the context

Govern — keep it all accountable

Where it all connects

Why the Sovereign Agentic OS exists

Artificial intelligence has learned to reason, write, and act. What it has not been given, in most organizations, is a safe place to do real work — a place where an agent can touch your data, your documents, and your systems without becoming a liability. Today that access is usually improvised: a script here, an API key there, a copied spreadsheet, a model with no idea who is asking or what they are allowed to see. That is not a foundation you can build a company on.

The **Sovereign Agentic OS** is the foundation. It is an EU-sovereign, open-source operating system that unifies **data, knowledge, connections, files, metrics, dashboards, software, agents, science, and strategy** into one governed platform — one you run on your own infrastructure (deployed on STACKIT), under your own control, subject to your own policy. It is not another dashboard bolted onto your stack. It is the operating layer beneath all of it.

Governance by default

Its defining idea is simple to state and hard to fake: **governance is not a feature, it is the default.** Every action in the OS — whether a person clicks a button or an agent calls a tool — runs **as the signed-in user**. It is checked against OPA policy, filtered by row-level and document-level security so you only ever see the rows and documents you are entitled to, and written to an immutable audit trail. Nothing runs as a nameless service account with god-mode access.

Crucially, **the agent interface is the same governed path as the UI**. When you connect Claude or another agent to the OS over the Model Context Protocol, it does not get a privileged back door. It travels the identical policy-checked, security-filtered, audited road that a human does — under *your* identity and *your* permissions. This is what makes it safe to hand an agent real work: it can only ever do what you could do, and every step is on the record.

Two cross-cutting models run through the entire OS. Learn these two ideas and the rest of the platform becomes legible.

The two ideas you must know

ROLES — FOUR RANKS, LOWEST TO HIGHEST

Authority in the OS is a simple ladder of four roles. Each rank contains the one below it and adds a well-defined power.

- **creator** — creates and runs their own work, and consumes what has been shared with them. The default rank for everyone doing the work.
- **builder** — a creator who can also *approve* Personal → Shared promotions within their domain. Builders are approvers of work, not administrators of people.
- **domain_admin** — a builder who can also administer users within their own domain: invite, assign roles up to builder, deactivate.

- **admin** — runs the whole tenant and certifies assets company-wide. The only role that can appoint a domain_admin.

The ladder is the same everywhere, so a permission you understand in one tab means the same thing in every other tab.

TIERS — FROM MY TO DOMAIN TO COMPANY

Every asset in the OS lives at one of three levels of visibility, and it always starts at the most private.

- **My** (*Personal*) — visible only to you. Everything begins here.
- **Domain** (*Shared*) — visible to your domain, reached by promotion with **builder** approval.
- **Company** (*Certified*) — trusted across the whole organization, reached by **admin** certification.

Promotion is never a silent flip of a switch. Moving an asset up a tier **always requires documentation first** — you describe what it is and why it can be trusted before anyone else comes to rely on it. That discipline is how trust scales from one person to a whole company without losing the thread of accountability.

How the OS is organised

The OS is arranged into five areas that mirror how work actually flows – from intent, to the context that governs it, to what you build, to how you keep it all accountable.

- **Plan** – set direction: strategy, initiatives, your operating model, and the workflows that run the business.
- **Context** – govern the knowledge and data your agents and people are allowed to use.
- **Build** – create agents, apps, models, and dashboards *on top of* that governed context.
- **Govern** – oversee and administer: approvals, monitoring, components, models, and platform administration.
- **Entry** – the front doors: orientation, your personal cockpit, tutorials, and the MCP connection.

The sections below walk every tab in order: a one-line reason it exists, and a short, practical note on how to use it.

Entry — the front doors

HOME

Why it exists: an orientation landing that lets you jump straight into any area of the OS. *How to use it:* start here when you are not sure where something lives. Home surfaces the five areas and the fastest route into each, so a new user can find their footing and an experienced one can move fast.

COCKPIT

Why it exists: your personal, at-a-glance operational overview. *How to use it:* open the Cockpit each morning to see pending work waiting on you — approvals, requests, runs — alongside recent activity. It is the single screen that answers "what needs me today?"

TUTORIALS

Why it exists: guided, in-product walkthroughs of the golden paths. *How to use it:* when you want to learn a workflow — building an agent, promoting a dataset — run the matching tutorial. It teaches by doing, inside the real product, rather than in a separate manual.

MCP

Why it exists: connects the OS to Claude and other agents over the Model Context Protocol, governed identically to the UI. *How to use it:* generate your per-user token and point your agent client at the OS. The agent then acts **as you**, bound by your permissions and fully audited. Configuring the connection surface is a builder-and-above task, but any user can connect and use their own token.

ABOUT / LICENSES

Why it exists: transparency about what you are running. *How to use it:* check here for the current version and the full set of open-source licenses behind the platform. Because the OS is open source, nothing about its composition is hidden from you.

Plan — set direction

STRATEGY

Why it exists: to hold your Strategic Pillars — long-horizon goals with a value metric, a target, a horizon, and progress so far. *How to use it:* define the handful of pillars that matter, each with a measurable outcome, and let everything downstream point back to them. Pillars are tiered My/Domain/Company and versioned, so direction is explicit, shared, and traceable over time.

BIG BETS

Why it exists: to plan the major initiatives that move a pillar. *How to use it:* tie each Big Bet to a pillar, then use the solution-design workspace to lay out the **workflow → the components that deliver it → their interplay → the context they use**, with value tracked against the goal. A Big Bet is where a strategic intention becomes a concrete, buildable plan.

OPERATING MODEL

Why it exists: the durable business context your agents build on. *How to use it:* maintain your operating model across **General • Strategy • Business • Organization • Architecture • Data • Glossary**, at My/Domain/Company scope. This is the stable, shared understanding of how the business works — the ground truth agents draw on so their output fits your reality, not a generic one.

WORKFLOWS

Why it exists: to capture business processes as living, governed artifacts. *How to use it:* model a process as a swimlane diagram with steps, actors, rules, and the know-how each step needs. Track readiness and gaps to see where a process is ready to automate, and export to PDF to share it. Workflows turn tacit process knowledge into something the OS — and its agents — can reason about.

MARKETPLACE

Why it exists: to discover and reuse company-certified assets. *How to use it:* browse Company-tier, certified assets — datasets, agents, dashboards — and adopt them instead of rebuilding. The Marketplace is how good, trusted work spreads across the organization. Available to builders and above.

Context — govern what agents may use

KNOWLEDGE

Why it exists: curated reference knowledge, written for both people and retrieval. *How to use it:* author reference material in markdown; the OS embeds it so it is RAG-retrievable by agents. Knowledge is foldered, versioned, and tiered My/Domain/Company — the deliberate, human-curated layer of what your agents know.

FILES

Why it exists: your unstructured documents, governed and retrievable. *How to use it:* upload documents and files into folders; the OS embeds them for retrieval so agents can ground answers in your real source material. Every file stays under the same governance and tiering as everything else.

DATA

Why it exists: structured data, refined through a disciplined pipeline. *How to use it:* bring data through a guided **Bronze → Silver → Gold** pipeline on the Trino lakehouse — raw, cleaned, then trusted. A catalog makes it discoverable, "Talk to Data" lets you query in natural language, and promotion moves gold data up the tiers for wider use.

CONNECTIONS

Why it exists: governed links to the external systems where your data already lives. *How to use it:* set up connectors to databases, warehouses, and SaaS tools — Slack, Gmail, GitHub, Snowflake, and more — grouped and searchable by category. Every connection is governed, so external reach never bypasses your policy.

METRICS

Why it exists: one agreed definition for each business metric. *How to use it:* define metrics on the Cube semantic layer so "revenue" or "active users" means exactly one thing everywhere. Query them, attach alerts, and promote the trusted ones. Metrics give agents and dashboards a single, governed source of numerical truth.

Build — create value on the context

AGENTS

Why it exists: to build multi-agent systems that do real work on your governed context. *How to use it:* move through five phases — **Define • Design • Build • Run • Evaluate**. Grant context to agents *as capabilities*, so an agent can only ever use what you have explicitly allowed. Then run it and evaluate the result, with downloadable reports for evidence. Governance is built into the build, not added after.

SOFTWARE

Why it exists: to build and deploy real applications on the platform. *How to use it:* create software that runs on the OS itself, with the same identity, policy, and audit guarantees as everything else. Your apps inherit governance rather than reinventing it.

SCIENCE

Why it exists: the full machine-learning lifecycle in one governed place. *How to use it:* **create → train → deploy → monitor** models without leaving the platform. Training runs, deployments, and monitoring all sit under the same policy and audit trail as your data and agents.

DASHBOARDS

Why it exists: business intelligence on your trusted data and metrics. *How to use it:* build BI dashboards embedded from the analytics engine on your gold data and defined metrics — with per-viewer row-level security, so each person sees only their entitled slice of the same dashboard. One dashboard, correctly filtered for everyone.

CONSOLE

Why it exists: a governed operator surface for direct queries. *How to use it:* run SQL over Trino and Cube in the Query console — every query policy- and RLS-checked per caller and audited, available to builders. A raw admin shell also lives here, but stays strictly admin-only. Power without a bypass.

Govern — keep it all accountable

POLICIES & APPROVALS

Why it exists: the governance inbox for the whole tenant. *How to use it:* review and approve promotions and requests, and see the policies currently in force. This is where Personal → Shared → Certified actually happens, with a person accountable at each gate. Available to builders and above.

MONITORING

Why it exists: observability across agents and the platform. *How to use it:* inspect agent runs, traces, cost, and logs to understand what happened and why. When an agent behaves unexpectedly, this is where you follow the thread. Available to builders and above.

COMPONENTS

Why it exists: the health of the platform's underlying services. *How to use it:* check the status of the OS's component services at a glance. An admin view for keeping the platform itself healthy.

LLM GATEWAY

Why it exists: self-service management of the models behind the OS. *How to use it:* manage models and providers through LiteLLM — add a provider, map it to a tier — so model choice stays configurable and never hardcoded. Available to builders and above.

ADMIN

Why it exists: platform administration for the tenant. *How to use it:* manage Users & Access, security, backups, and tenant settings. Full administration is an admin task; a builder who visits sees only their own Settings, never the levers of the tenant.

Where it all connects

The five areas are not silos — they are a single loop. **Plan** sets the direction and names the outcomes that matter. **Context** governs exactly what data and knowledge may be used in pursuit of them. **Build** creates the agents, apps, models, and dashboards that turn that context into value. And **Govern** keeps every step of it accountable — who approved what, what ran, what it cost, and who could see it.

The thread running through all of it is the same one you met at the start: every action runs *as you*, checked, filtered, and recorded — whether a person or an agent takes it. That is what lets you give AI real hands on your work without giving up control of it.

To get started, open the **Cockpit** to see what is waiting on you, and run a **Tutorial** to walk a golden path end to end. Everything else is the same operating model, applied one tab at a time.